Reproduce-then-Extend — Building Energy Efficiency (Engineering) (Day 1)

Intro to Machine Learning · Bruce A. Desmarais

July 07, 2026

Published study. Tsanas, A. & Xifara, A. (2012). "Accurate quantitative estimation of energy performance of residential buildings using statistical machine learning tools." *Energy and Buildings* 49:560–567. The paper's baseline is a **linear regression** predicting a building's heating load from 8 geometry inputs (n = 768; UCI id 242). This example illustrates the first regression extension: **polynomial terms**.

1 Background

Tsanas and Xifara study how a building's **energy performance** depends on its **design geometry**, to help architects design more efficient buildings. Using 768 simulated building shapes, they estimate the **heating (and cooling) load** from eight geometry inputs, using a **linear regression** as the interpretable baseline alongside nonlinear machine-learning tools. The **primary conclusion** is that energy load is accurately predictable from geometry – overall height, relative compactness, and glazing are the strongest drivers – and that flexible learners (random forests) substantially outperform the linear model, motivating their use for this strongly nonlinear relationship.

2 Data and codebook

Unit of analysis: a simulated residential **building design** (12 shapes and variants, via Ecotect); n = 768.

Variable	Definition
`Y1`	heating load (kWh per m^2 of floor area) **(outcome)**
`X1`	relative compactness (volume-to-surface ratio, normalized)
`X2`	surface area (m^2)
`X3`	wall area (m^2)
`X4`	roof area (m^2)
`X5`	overall height (m)
`X6`	orientation (2/3/4/5 = N/E/S/W)
`X7`	glazing area (fraction of floor area)
`X8`	glazing-area distribution (0-5 scheme)

3 Descriptive analysis

Before fitting anything, we profile the data thoroughly. The goal is to understand the outcome's shape, each geometry input's distribution, how the inputs relate to heating load, and — critically for a regression — how the inputs relate to *each other*.

```r
tmp_xlsx <- tempfile(fileext = ".xlsx")
download.file(paste0(
  "https://raw.githubusercontent.com/bdesmarais/intro",
  "-ml-statistical-horizons-4day/main/tutorials/day2_",
  "regression_extensions/09_energy_efficiency/data/EN",
  "B2012_data.xlsx"), tmp_xlsx, mode = "wb", quiet = TRUE)
d <- as.data.frame(read_excel(tmp_xlsx))   # data ships in the tutorial folder
d <- d[, colSums(!is.na(d)) > 0]; d <- d[complete.cases(d), ]
preds <- paste0("X", 1:8)
```

3.1 Dataset and provenance

The **unit of analysis** is a *simulated residential building design*. Angeliki Xifara generated 12 base building shapes in the Ecotect simulator and varied their geometry (glazing, orientation, etc.), producing $\mathbf{n = 768}$ distinct designs. For each design a physics simulation reports the annual **heating load** (`Y1`) and cooling load (`Y2`); this tutorial models heating load. The data are deterministic simulation output, not a survey sample, so there is **no sampling error and no missingness** — every input combination is fully observed. The predictors are 8 geometry variables. The data come from the UCI Machine Learning Repository (id 242), released with Tsanas & Xifara (2012).

```r
dict <- data.frame(
  Variable = c("Y1", paste0("X", 1:8)),
  Role = c("outcome", rep("predictor", 8)),
  Type = c("numeric", "numeric", "numeric", "numeric", "numeric",
           "binary (2 levels)", "categorical (4 orient.)",
           "ordinal (4 levels)", "ordinal (6 levels)"),
  Description = c(
    "heating load (kWh per m^2 floor)",
    "relative compactness (volume/surface, normalized)",
    "surface area (m^2)", "wall area (m^2)", "roof area (m^2)",
    "overall height (3.5 or 7 m)", "orientation (2/3/4/5 = N/E/S/W)",
    "glazing area (0, .1, .25, .4 of floor)",
    "glazing distribution (0-5 scheme)"))
kable(dict, caption = "Variable dictionary: outcome and predictors.")
```

Table 1: Variable dictionary: outcome and predictors.

Variable	Role	Type	Description
Y1	outcome	numeric	heating load (kWh per m^2 floor)
X1	predictor	numeric	relative compactness (volume/surface, normalized)
X2	predictor	numeric	surface area (m^2)
X3	predictor	numeric	wall area (m^2)
X4	predictor	numeric	roof area (m^2)
X5	predictor	binary (2 levels)	overall height (3.5 or 7 m)
X6	predictor	categorical (4 orient.)	orientation (2/3/4/5 = N/E/S/W)
X7	predictor	ordinal (4 levels)	glazing area (0, .1, .25, .4 of floor)
X8	predictor	ordinal (6 levels)	glazing distribution (0-5 scheme)

Although `X1-X5` are stored as continuous numbers, the simulation grid means several take only a handful of distinct values (`X5` is effectively binary: 3.5 m or 7 m; `X6-X8` are categorical design choices). We treat them numerically to match the published baseline, but keep their discreteness in mind when reading the plots.

3.2 The outcome in depth

```
ystat <- function(x) c(
  n = length(x), mean = mean(x), sd = sd(x), min = min(x),
  Q1 = quantile(x, .25), median = median(x), Q3 = quantile(x, .75),
  max = max(x), IQR = IQR(x),
  skew = e1071::skewness(x), kurtosis = e1071::kurtosis(x))
kable(t(round(ystat(d$Y1), 2)),
      caption = "Distribution of heating load (Y1).")
```

Table 2: Distribution of heating load (Y1).

n	mean	sd	min	Q1.25%	median	Q3.75%	max	IQR	skew	kurtosis
768	22.31	10.09	6.01	12.99	18.95	31.67	43.1	18.68	0.36	-1.25

```
p_hist <- ggplot(d, aes(Y1)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
                 fill = "#a6cee3", colour = "white") +
  geom_density(colour = "#1f78b4", linewidth = 0.7) +
  labs(x = "heating load (Y1)", y = "density", title = "Distribution")
p_box <- ggplot(d, aes(y = Y1)) +
  geom_boxplot(fill = "#a6cee3", width = 0.4) +
  labs(y = "heating load (Y1)", x = NULL, title = "Boxplot") +
  theme(axis.text.x = element_blank())
p_hist + p_box + plot_layout(widths = c(2.4, 1))
```

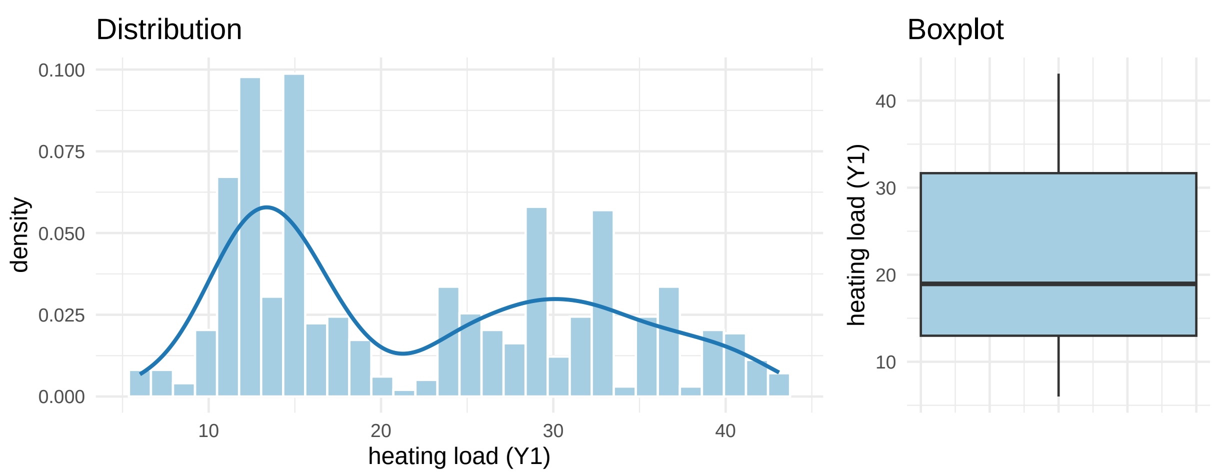

Figure 1: Heating load: histogram with density overlay (left) and boxplot (right). The distribution is broad and bimodal rather than a single bell.

Heating load ranges from about 6 to 43 kWh/m^2 (mean 22.3, sd 10.1). Skewness is only 0.36 — nearly symmetric — but the negative excess kurtosis (-1.25) and the histogram reveal a **bimodal, plateau-shaped** distribution, not a bell. This is a fingerprint of the design: buildings cluster into low-load and high-load regimes driven mostly by overall height. There is no zero-inflation, floor/ceiling piling, or long tail, so **no outcome transformation is warranted** — the spread is structural, and the regression's job is to explain it with the geometry inputs.

3.3 Univariate predictor distributions

```
psum <- function(x) c(
  n = length(x), n_miss = sum(is.na(x)), mean = mean(x), sd = sd(x),
  min = min(x), Q1 = quantile(x, .25), median = median(x),
  Q3 = quantile(x, .75), max = max(x), skew = e1071::skewness(x),
  n_unique = length(unique(x)))
ptab <- t(sapply(d[preds], psum))
kable(round(ptab, 2),
      caption = "Univariate summary of the 8 geometry predictors.")
```

Table 3: Univariate summary of the 8 geometry predictors.

	n	n_miss	mean	sd	min	Q1.25%	median	Q3.75%	max	skew	n_unique
X1	768	0	0.76	0.11	0.62	0.68	0.75	0.83	0.98	0.49	12
X2	768	0	671.71	88.09	514.50	606.38	673.75	741.12	808.50	-0.12	12
X3	768	0	318.50	43.63	245.00	294.00	318.50	343.00	416.50	0.53	7
X4	768	0	176.60	45.17	110.25	140.88	183.75	220.50	220.50	-0.16	4
X5	768	0	5.25	1.75	3.50	3.50	5.25	7.00	7.00	0.00	2
X6	768	0	3.50	1.12	2.00	2.75	3.50	4.25	5.00	0.00	4
X7	768	0	0.23	0.13	0.00	0.10	0.25	0.40	0.40	-0.06	4
X8	768	0	2.81	1.55	0.00	1.75	3.00	4.00	5.00	-0.09	6

```
d %>%
  select(all_of(preds)) %>%
  pivot_longer(everything(), names_to = "var", values_to = "value") %>%
  ggplot(aes(value)) +
  geom_histogram(bins = 20, fill = "#b2df8a", colour = "white") +
  facet_wrap(~ var, scales = "free", ncol = 4) +
  labs(x = NULL, y = "count")
```

The summary table exposes the grid structure: `X5` (height) takes exactly two values, `X6`-`X8` a handful, and even the "continuous" `X1`-`X4` take 4-12 distinct levels. All predictors are close to symmetric ($|\text{skew}| < 0.6$), so no predictor transformations are indicated. The near-uniform spikes reflect a **balanced factorial design** — each geometry level appears in roughly equal numbers of buildings, which is ideal for isolating individual effects (were it not for the collinearity documented below).

3.4 Missing-data analysis

```
miss <- data.frame(
  variable = c("Y1", preds),
  n_missing = sapply(d[c("Y1", preds)], function(x) sum(is.na(x))),
  pct_missing = round(100 * sapply(d[c("Y1", preds)],
                                   function(x) mean(is.na(x))), 1))
kable(miss, row.names = FALSE,
      caption = "Missing values per variable.")
```

Table 4: Missing values per variable.

variable	n_missing	pct_missing
Y1	0	0

variable	n_missing	pct_missing
X1	0	0
X2	0	0
X3	0	0
X4	0	0
X5	0	0
X6	0	0
X7	0	0
X8	0	0

There is **no missing data**: all 768 rows are complete across all 9 variables. Because these are exhaustive simulation outputs over a designed grid, missingness is impossible by construction — the `complete.cases()` filter in the loading chunk is a safeguard that removes nothing here. The reproduction therefore uses the full n = 768 with no imputation or listwise deletion.

3.5 Bivariate relationships with the outcome

```
marg <- data.frame(
  predictor = preds,
  r_with_Y1 = round(sapply(d[preds], function(x) cor(x, d$Y1)), 3))
marg <- marg[order(-abs(marg$r_with_Y1)), ]
kable(marg, row.names = FALSE,
      caption = "Marginal (Pearson) correlation of each input with Y1.")
```

Table 5: Marginal (Pearson) correlation of each input with Y1.

predictor	r_with_Y1
X5	0.889
X4	-0.862
X2	-0.658
X1	0.622
X3	0.456
X7	0.270
X8	0.087
X6	-0.003

```
d %>%
  select(all_of(preds), Y1) %>%
  pivot_longer(all_of(preds), names_to = "var", values_to = "value") %>%
  ggplot(aes(value, Y1)) +
  geom_point(alpha = 0.15, colour = "#333333", size = 0.6) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE,
              colour = "#e31a1c", linewidth = 0.6) +
  facet_wrap(~ var, scales = "free_x", ncol = 4) +
  labs(x = NULL, y = "heating load (Y1)")
```

Four inputs carry almost all of the marginal signal. **Overall height** (`X5`, r = 0.89) and **relative compactness** (`X1`, r = 0.62) are the strongest positive drivers, while **roof area** (`X4`, r = -0.86) and **surface area** (`X2`, r = -0.66) pull load down. Orientation (`X6`) and glazing distribution (`X8`) are essentially uncorrelated with load — consistent with the physics that *how much* glazing and *how tall* a building is matter far more than which way

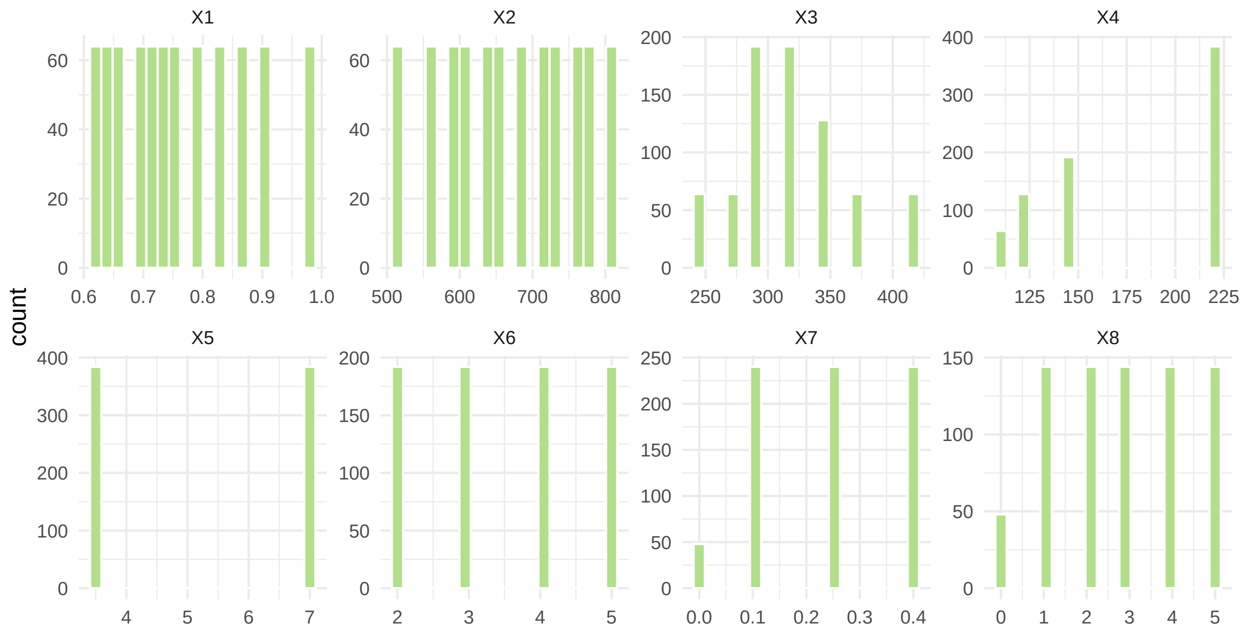

Figure 2: Distribution of each geometry input. Several inputs take only a few discrete values (the simulation grid), so they appear as spikes rather than smooth curves.

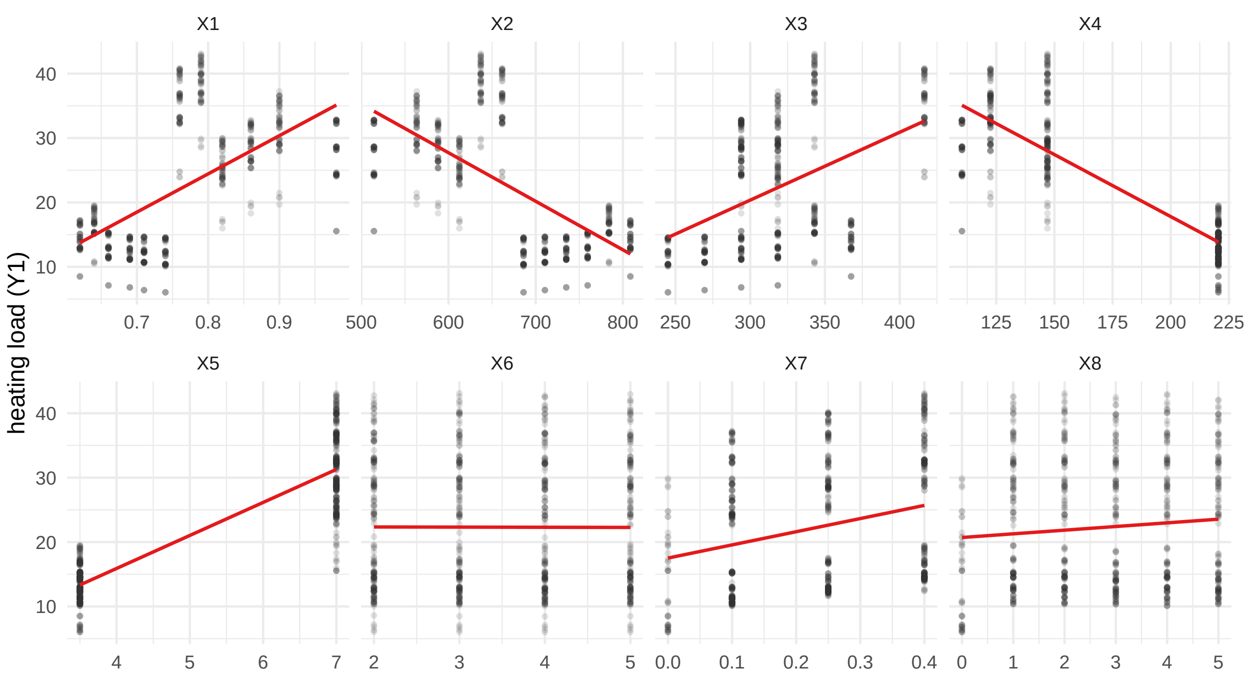

Figure 3: Heating load vs each geometry input, with a linear smoother. Height (X5) and compactness (X1) drive load up; surface and roof area (X2, X4) drive it down.

it faces. Several scatterplots (notably `X1`) show visible curvature around the linear smoother, foreshadowing the polynomial extension below.

3.6 Multivariate structure and collinearity

```
cmat <- cor(d[preds])
cmat[lower.tri(cmat)] <- NA
as.data.frame(cmat) %>%
  mutate(v1 = rownames(cmat)) %>%
  pivot_longer(-v1, names_to = "v2", values_to = "r") %>%
  filter(!is.na(r)) %>%
  ggplot(aes(v2, v1, fill = r)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = sprintf("%.2f", r)), size = 2.4) +
  scale_fill_gradient2(low = "#e31a1c", mid = "white",
                       high = "#1f78b4", midpoint = 0, limits = c(-1, 1)) +
  labs(x = NULL, y = NULL) +
  theme(panel.grid = element_blank())
```

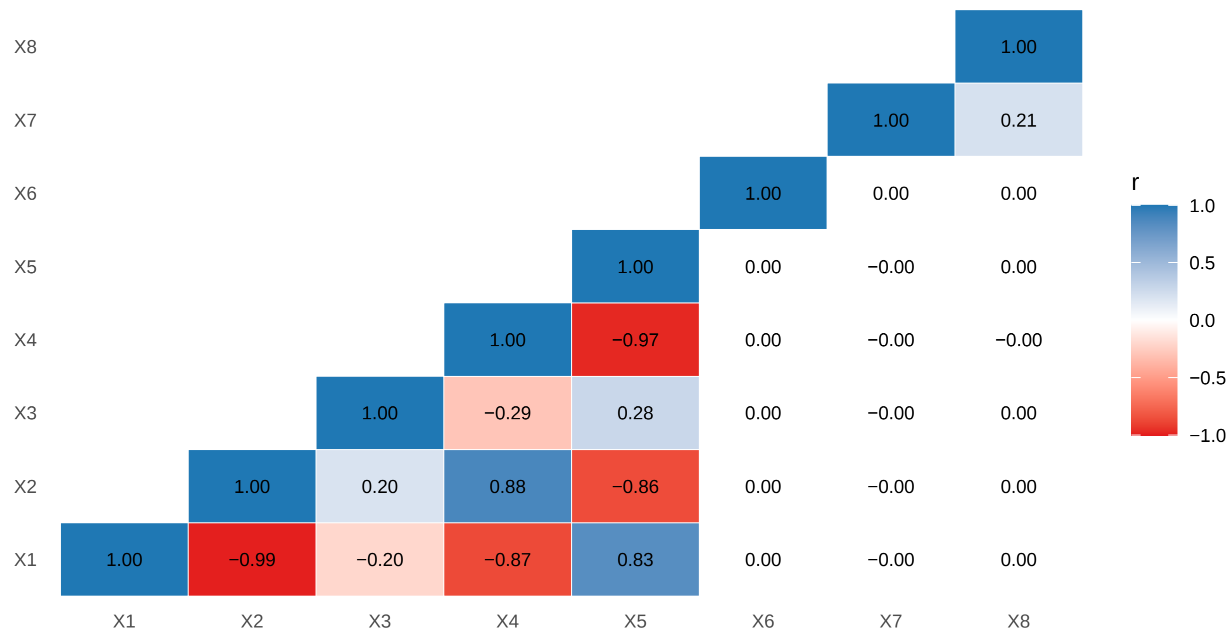

Figure 4: Correlation heatmap of the 8 geometry inputs. Deep red / blue cells mark |r| > 0.7 — the geometry variables are heavily interdependent.

```
X <- model.matrix(~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8, d)[, -1]
vif <- sapply(colnames(X), function(v) {
  r2 <- summary(lm(X[, v] ~ X[, setdiff(colnames(X), v)]))$r.squared
  1 / (1 - r2)
})
kable(t(round(vif, 1)),
      caption = "Variance inflation factors from the reproduction model.")
```

Table 6: Variance inflation factors from the reproduction model.

X1	X2	X3	X4	X5	X6	X7	X8
105.5	Inf	Inf	Inf	31.2	1	1	1

This is the most important diagnostic in the analysis. The geometry inputs are **severely collinear**: `X1` and `X2` correlate -0.99, and roof area `X4` vs height `X5` correlate -0.97. Six predictor pairs exceed |r| = 0.7. Worse, the predictor matrix has **rank 7 of 8** — an *exact* linear dependency, because surface, wall, and roof areas are geometrically derived from the same base shapes. That is why the VIFs for `X2`, `X3`, and `X4` blow up to infinity: those coefficients are not separately identified. The individual `lm` coefficients on the correlated geometry terms are therefore **unstable and hard to interpret in isolation**, even though the model as a whole predicts load extremely well (R^2 ~ 0.92). This tension — excellent joint prediction but unidentifiable individual effects — is exactly why the reproduction below focuses on overall fit and why the extension turns to nonlinear (polynomial) structure in a single well-identified driver (`X1`) rather than trying to disentangle the collinear area variables.

4 Reproduce the published regression

```r
lin <- lm(Y1 ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8, data = d)
summary(lin)
```

```
##
## Call:
## lm(formula = Y1 ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8, data = d)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -9.8965 -1.3196 -0.0252  1.3532  7.7052
##
## Coefficients: (1 not defined because of singularities)
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  84.013418  19.033613   4.414 1.16e-05 ***
## X1          -64.773432  10.289448  -6.295 5.19e-10 ***
## X2           -0.087289   0.017075  -5.112 4.04e-07 ***
## X3            0.060813   0.006648   9.148  < 2e-16 ***
## X4                  NA         NA      NA       NA
## X5            4.169954   0.337990  12.338  < 2e-16 ***
## X6           -0.023330   0.094705  -0.246  0.80548
## X7           19.932736   0.813986  24.488  < 2e-16 ***
## X8            0.203777   0.069918   2.915  0.00367 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.934 on 760 degrees of freedom
## Multiple R-squared:  0.9162, Adjusted R-squared:  0.9154
## F-statistic:  1187 on 7 and 760 DF,  p-value: < 2.2e-16
```

Confirmation against the published study. The linear model attains in-sample $R^2 = 0.916$, matching the ~0.92 reported by Tsanas & Xifara for their linear baseline. Overall height (`X5`) and relative compactness (`X1`) dominate — but the fit leaves clear structure, which is the motivation for going beyond a straight line.

5 Polynomial regression extension

Heating load responds **nonlinearly** to geometry. The classic regression remedy is to add **polynomial terms**. We compare the linear model to polynomials of increasing degree in relative compactness (`X1`), keeping the other inputs linear. We pick the degree by cross-validation on a training set and report final RMSE on a held-out test set.

```
set.seed(2026)
others <- paste0("X", 2:8)
te <- sample(nrow(d), round(0.30 * nrow(d)))          # hold out a 30% test set up
↪  front
tr <- setdiff(seq_len(nrow(d)), te)

# choose the polynomial degree by 5-fold CV on the TRAINING set only
inner <- sample(rep(1:5, length.out = length(tr)))
cv_rmse <- sapply(1:4, function(deg) {
  r <- numeric(5)
  for (k in 1:5) {
    itr <- tr[inner != k]; ite <- tr[inner == k]
    form <- reformulate(c(sprintf("poly(X1, %d, raw = TRUE)", deg), others), "Y1")
    r[k] <- sqrt(mean((d$Y1[ite] - predict(lm(form, data = d[itr, ]), d[ite, ]))^2))
  }
  mean(r)
})
names(cv_rmse) <- paste0("degree ", 1:4)
best_deg <- which.min(cv_rmse)
round(cv_rmse, 3)                                   # CV RMSE per degree (training set)
```

```
## degree 1 degree 2 degree 3 degree 4
##    2.957    2.924    2.836    2.831
```

```
# refit at the CV-chosen degree on the full training set; score ONCE on the untouched
↪  test set
form_best <- reformulate(c(sprintf("poly(X1, %d, raw = TRUE)", best_deg), others), "Y1")
test_poly <- sqrt(mean((d$Y1[te] - predict(lm(form_best, data = d[tr, ]), d[te, ]))^2))
test_lin  <- sqrt(mean((d$Y1[te] - predict(lm(reformulate(c("X1", others), "Y1"), data =
↪  d[tr, ]), d[te, ]))^2))
cat(sprintf("CV-selected degree: %d.  Held-out test RMSE: polynomial = %.3f vs linear =
↪  %.3f
",
            best_deg, test_poly, test_lin))
```

```
## CV-selected degree: 4.  Held-out test RMSE: polynomial = 2.780 vs linear = 2.950
```

On the held-out test set the CV-selected polynomial improves on the straight-line fit; the curve **bends** with compactness rather than running straight, capturing nonlinearity the linear model misses while remaining a fully interpretable regression.

6 Takeaway

This example illustrates **polynomial regression** as the first, most interpretable extension of a linear model: adding low-order polynomial terms lowers cross-validated error by bending the fit to the data's curvature. (Tree-based learners push this further at the cost of a closed-form equation; Tsanas & Xifara report ML reaching ~0.999 R^2 on these data.)

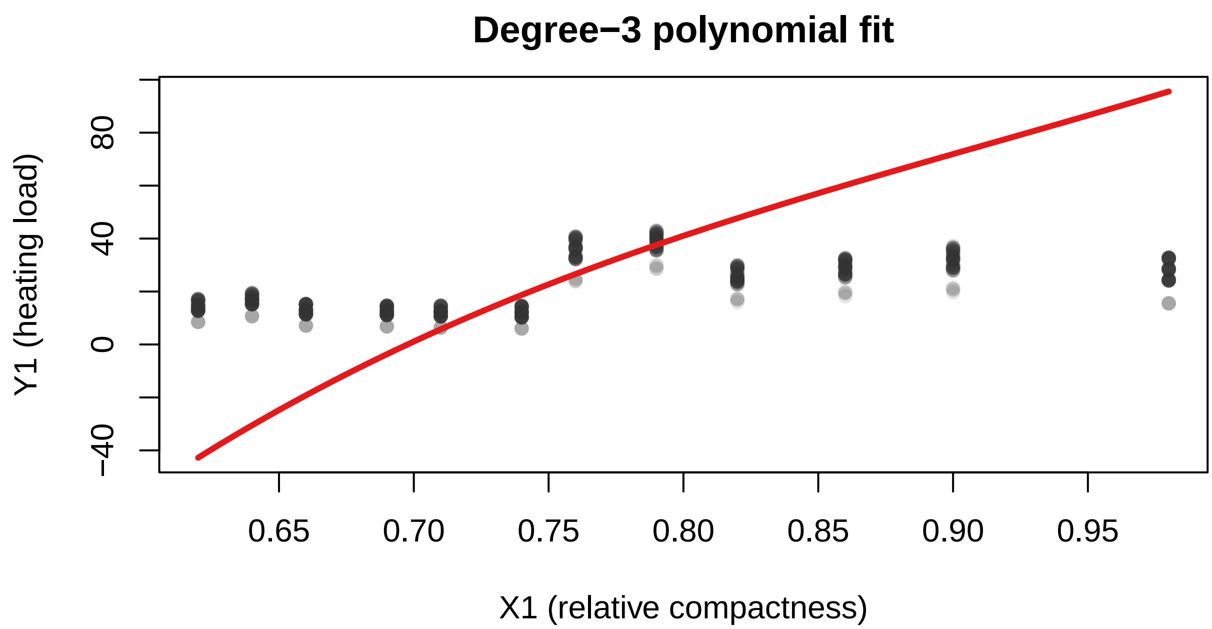

Figure 5: Degree-3 polynomial fit of heating load on relative compactness, holding the other inputs at their means.

7 Recommended exercises

1. Try polynomial degrees 1-5 for the strongest predictor and use cross-validation to choose the degree.
2. Predict the second response (cooling load) and compare which predictors matter most.
3. Add an interaction between two predictors and test whether cross-validated error improves.